



Phyton endterm cheat-sheet

Introduction to python (Universiteit van Amsterdam)



Scan to open on Studocu

Suppose you have three variables **x**, **y**, **z**, as follows:

```
x = 3
y = 3
z = 3.0
```

What are the outputs of the following lines of code?

```
print(x == y)
print(y == z)
print(x == z)
print(x == y == z)
```

ANS:
False
True
False
False

def calculation(func, *args):

```
    result = 0
    for el in args:
        result += func(el)
    return result
```

Which of the following function calls would return the number 30 as a result?

ANS:
calculation(lambda x: x**2, 1, 2, 3, 4)
calculation(lambda x: x + 5, 1, 2, 3, 4)

You have a list of strings called **list_1**, in which every element is unique.

Which of the following code lines will create a dictionary, in which the keys are the elements of **list_1** and the values are the number of characters in the corresponding key?

ANS:
{item: len(item) for item in list_1}
Suppose that you have a list of letters called **list_1**. Some letters are upper case, others are in lower case.

You'd like to create another list that contains the same letters in the same order, but you want to turn every upper case letter into its lower case version, and every lower case letter into its upper case version.

Which of the following code lines will achieve what you want?

ANS:
{letter.upper() if letter.islower() else letter.lower() for letter in list_1}

class Book:

```
    def __init__(self, title, author, review_scores=None):
        self.title = title
        self.author = author
        self.review_scores = review_scores if review_scores else []
    def add_review(self, score):
        self.review_scores.append(score)
    def show_rating(self):
        if self.review_scores:
            return round(sum(self.review_scores) / len
(self.review_scores), 1)
```

Suppose that we create a **Book** object and call its **add_review** method three times, like this:

```
book_1 = Book("The Lightning Thief", "Rick Riordan")
book_1.add_review(5)
book_1.add_review(3)
book_1.add_review(3)
```

What will then the following command print to the screen?

```
print(book_1.show_rating())
```

ANS: 3.7

You'd like to define a class called **Vehicle**. It should have two attributes: "name" and "mode", which you want to be initialized at the time of object construction.

The "name" attribute can be any string, and it must be passed to the object constructor. The "mode" attribute is also a string, but it doesn't necessarily need to be passed to the object constructor. If it isn't specified at object construction, then the "mode" attribute should be equal to "land".

For example, if we create a **Vehicle** object as:

```
my_car = Vehicle("Mazda")
then the following two conditions should both be true:
my_car.name == "Mazda"
my_car.mode == "land"
```

ANS:

class Vehicle:

```
    def __init__(self, vehicle_name, vehicle_mode="land"):
        self.name = vehicle_name
        self.mode = vehicle_mode
```

You need to write a simple loop that iterates over the integers from 1 to 10, and in each iteration, prints the sum of the current and the previous number. (In the first iteration, just take the previous number to be 0.)

For example:

```
1 + 0 = 1
2 + 1 = 3
3 + 2 = 5
10 + 9 = 19
```

Choose the program that will print the correct output (i.e.: only the sums)

ANS:
previous = 0
for i in range(1, 11):
 print(i + previous)
 previous = i

You need to write a function called **format_phone_number** that formats phone numbers. The output of the function should be a string that represents a 10-digit number. The input argument is also a string with 10 digits, but the digits may be separated by dashes (the "-" character) at various places.

For example, the following function calls:
format_phone_number("020-525-1400")
format_phone_number("020-5251400")
format_phone_number("020-525-14-00")
should all return the string:
"0205251400"

Which of the following code segments does achieve what you want?

ANS:
def format_phone_number(number):
 return "".join(number.split("-"))
def format_phone_number(number):
 return number.replace("-", "")

Suppose that you'd like to create a new DataFrame that only contains those elements of column "B" in **df** that have an even-numbered index. Which of the following code lines will achieve this?

ANS: df.loc[df.index % 2 == 0, ["B"]]

Executing the following program will produce an error. Why?

```
def cube(num):
    result = num ** 3
    return f'The cube of {num} is: {result}.'
print(result)
```

ANS:
Result is a local variable and cannot be referenced outside of the function.

list_1 = [lambda a, b: a + b, lambda a, b: a * b]
What would be printed by the following code snippet:
print(list_1[0](1, 2) ** list_1[1](1, 2))

ANS: 9
day_of_year(datetime(2022, 2, 1))
should return the integer:
32

ANS:
def day_of_year(dt):
 return (dt - datetime(dt.year, 1, 1)).days + 1

x = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']
then how do you create:

y = [['a', 'b', 'c'], ['d', 'e', 'f'], ['g', 'h', 'i']]

ANS:
y = [x[i+3] for i in range(0, 7, 3)]
students = [{"Name": "Adam", "Grade": 7.5}, {"Name": "Bernard", "Grade": 8.0}]

then you'd like to see the following two lines printed to the screen:

Adam has received a grade of 7.5.
Bernard has received a grade of 8.0.

ANS:
for student in students:
 print(f'{student["Name"]} has received a grade of {student["Grade"]:.1f}.')

def main(x):
 y = []
 for i in x:
 if len(i) == 0:
 y.append(True)
 else:
 y.append(False)
 return y[0] == y[1] == y[2]

What are the outputs of the following lines:

```
main([[], [], []])
main([[], [2, 2], [3, 3, 3]])
main([[], [2], [3]])
```

ANS: True x3
There is a pandas Series called **s**, which contains Dutch municipality and corresponding province names, separated by a semi-colon and a whitespace. The following, for example, is an element of this Series: "Amsterdam; Noord-Holland".
You'd like to create a new Series that only contains the municipality names from **s** in the same order. Which of the following code lines will achieve what you want?

ANS:
s.map(lambda x: x.split(";")[0])

```
l1 = [1, 2, 3, 4]
l2 = [2, 3, 4, 5]
```

```
d1 = {}
for key, value in zip(l1, l2):
    d1[key] = value
d2 = {}
for key, value in enumerate(l2, 1):
    d2[key] = value
print(d1 == d2)
```

ANS: True
Return another dictionary that has the same keys as the input argument. For each output key, the corresponding value should be calculated as the sum of those values from the input dictionary, whose keys are smaller than, or equal to, the output key in question.

For example:
main({5: 1, 2: 5, 4: 2})
should return a dictionary that is equal to:
{5: 8, 2: 5, 4: 7}

ANS:
def main(d1):
 running_total = 0
 d2 = {}
 for key in sorted(d1.keys()):
 running_total += d1[key]
 d2[key] = running_total
 return d2

ANS 2:
def main(d):
 result = {}
 for output_key in d.keys():
 result[output_key] = sum([v for k, v in d.items() if k <= output_key])
 return result

A column called "C", which contains the sum of the numbers in columns "A" and "B". Which of the following code lines will achieve what you want?

ANS:
df["C"] = df["A"] + df["B"]
A C D
4 7.8 7.9 8.0
3 3.2 8.9 3.6
2 6.0 9.1 3.3
ANS:
df2 = df1.iloc[[3, 2, 1], [0, 2, 3]]
or
df2 = df1.loc[2:4, ["A", "C", "D"]].sort_values("C")

def get_months(date_strings):
 dates = []
 for date_str in date_strings:
 if "-" in date_str:
 dates.append(datetime.strptime(date_str, "%d-%m-%Y"))
 elif "/" in date_str:
 dates.append(datetime.strptime(date_str, "%m/%d/%Y"))
 else:
 dates.append(None)
 return [date.month if date is not None else None for date in dates]
Suppose that you call the function like this:
get_months(["03/02/2013", "03-02-2013", "03-02-2013"])

ANS: [3, None, 2]

Print the label of the element that belongs to the value 20. For example: if, x = ['A', 10, 'B', 20, 'C', 30, 'D', 40] your program should print: B
ANS: print(x[x.index(20) - 1])

f-string:

```
>>> greeting = "Hello"
>>> first_name = "John"
>>> print(f'{greeting}, my name is {first_name}!')
Hello, my name is John!
```

.split():

```
>>> string_6 = "They are at the shop."
>>> print(string_6.split())
['They', 'are', 'at', 'the', 'shop,']
>>> print(string_6.split("at"))
['They are ', ' the shop.']
```

Nested lists:

```
>>> list_4 = [[1, 2], ['a', 'b', 'c']] # outer list
>>> print(list_4[0]) # accessing inner list
>>> print(list_4[0][1]) # accessing inner list item
[1, 2]
2
```

.items() in dictionary

The items() method returns view object, that displays all key-value pairs of the dictionary, as tuples in a list. So, if we want to know the key-value pairs of the applicant's grades, then we can apply this method.

```
>>> print(grade_list.items())
dict_items([('Mathematics', 9.5), ('Literature', 7.5), ('Physics', 8.0), ('Chemistry', 6.0), ('Arts', 7.0)])
```

range() function:

```
range(start, stop, step_size)
>>> print(list(range(5)))
>>> print(list(range(2, 6)))
>>> print(list(range(2, 12, 3)))
[0, 1, 2, 3, 4] [2, 3, 4, 5] [2, 5, 8, 11]
```

Zip() function:

```
>>> students = ["Adrian", "Benjamin", "Carol"]
>>> problem_number = [1, 2, 3]
>>> student_and_problem = zip(students, problem_number)
```

```
>>> print(student_and_problem)
<zip object at 0x7fdd6fba9b90>
>>> print(list(student_and_problem))
[('Adrian', 1), ('Benjamin', 2), ('Carol', 3)]
```

to unzip:

```
>>> students_name, students_height = zip
(*students_and_height)
```

List comprehension

```
>>> list_1 = [1, 2, 3, 4, 5]
>>> list_even = [num for num in list_1 if not num % 2]
>>> print(list_even)
[2, 4]
>>> [num ** 2 if num % 2 == 0 else 0 for num in range(10)]
[0, 0, 4, 0, 16, 0, 36, 0, 64, 0]
```

Functions:

```
>>> def welcome_customer(name):
>>>     print(f'Welcome {name} at Eat & Chill, enjoy your stay with us!')
>>> welcome_customer("Andrew")
Welcome Andrew at Eat & Chill, enjoy your stay with us!
```

Datetime:

```
.strftime()
>>> from datetime import datetime
>>> exam_date = "27/05/2021"
>>> print(exam_date)
>>> # create datetime object, representing it with " dt"
>>> exam_date_dt = datetime.strptime(exam_date, "%d/%m/%Y")
>>> print(exam_date_dt)
27/05/2021
2021-05-27 00:00:00
>>> .strftime() does the opposite
>>> print(exam_date_dt.strftime("%Y/%m/%d"))
2021/05/27
```

Timezones:

```
>>> from datetime import datetime
>>> from pytz import timezone
>>> record_dt = datetime.strptime('07/12/2016 04:39PM', "%m/%d/%Y %H:%M%p")
>>> # defining the timezones
>>> new_york_tz = timezone('US/Eastern')
>>> los_angeles_tz = timezone('US/Pacific')
>>> new_york_dt = record_dt.replace(tzinfo = new_york_tz)
>>> # change the timezone of New York as if it was in Los Angeles (2 hour s)
>>> los_angeles_dt = new_york_dt.astimezone(los_angeles_tz)
>>> print(new_york_dt)
>>> print(los_angeles_dt)
2016-07-12 00:00:00-04:56 \ 2016-07-11 21:56:00-07:00
```

Timedelta:

```
>>> from datetime import datetime
>>> from datetime import timedelta
>>> invoice_issue_date = datetime(2019, 5, 13)
>>> payment_term = timedelta(days = 25)
>>> invoice_end_date = invoice_issue_date + payment_term
>>> print(invoice_end_date) // 2019-06-07 00:00:00
Pendulum:
>>> import pendulum
>>> election = "2021-10-11" # parse datetime object by pendulum and set as aware for timezone, US/ Eastern
>>> election_pend = pendulum.parse(election, tz="US/ Eastern")
>>> today = pendulum.now("Asia/Tokyo")
>>> difference = election_pend - today
>>> print("days:", difference.in_days()) // days: 146
>>> print("months:", difference.in_months()) // months: 4
```

Classes:

```
>>> class Student:
>>>     def degree_level(self, level):
>>>         print(f"my current degree level is {level}")
>>> student_1 = Student()
>>> student_1.degree_level("bachelor")
my current degree level is bachelor
```

```
>>> class Student():
>>>     def set_first_name(self, first_name):
>>>         self.first_name = first_name
>>>     def set_last_name(self, last_name):
>>>         self.last_name = last_name
>>>     def get_full_name(self):
>>>         return f"{self.first_name} {self.last_name}"
>>> student_1 = Student()
>>> # to set certain names to the student
>>> student_1.set_first_name("James")
>>> student_1.set_last_name("Davis")
>>> student_1.get_full_name() // 'James Davis'
```

```
> _init_
> class Student():
>>>     def _init_(self, first_name, last_name, degree_level,
>>>         status, st udy_year):
>>>         self.first_name = first_name
>>>         self.last_name = last_name
>>>         self.degree_level = degree_level
>>>         self.status = status
>>>         self.study_year = study_year
>>> student_1 = Student("James", "Davis", "bachelor",
>>>     "enrolled", "2")
```

.sort_index() in DataFrame

```
>>> import pandas as pd
>>> df = pd.read_csv("iris.csv")
>>> print(df.sort_index(axis=1, ascending = False).head())
```

.sort_values()

```
>>> import pandas as pd
>>> df = pd.read_csv("iris.csv")
>>> print(df.sort_values(by = "sepal_length", ascending =
>>>     True).head())
```

Selection and conditional selection:

```
print(df["petal_length"].head())
print(df[0:4].head())
print(df["sepal_length"] >= 4.87)
```

Isoformat:

```
print(datetime.isoformat("|") // 2017-02-15|20:26:08.937881
```

```
df.loc
      cobra      max_speed  shield
viper          4            5
sidewinder      7            8
```

```
>>> df.loc['viper']      >>> df.loc[['viper', 'sidewinder']]
max_speed      4      max_speed  shield
shield          5      sidewinder      7      8
```

```
>>> df.loc['cobra', 'shield']
2
```

```
df.iloc
      a      b      c      d
0      1      2      3      4
1     100     200     300     400
2     1000    2000    3000    4000
```

```
>>> df.iloc[[0, 1]]
      a      b      c      d
0      1      2      3      4
1     100     200     300     400
```

```
>>> df.iloc[[0]]
      a      b      c      d
0      1      2      3      4
```

Use comprehension to compile a new list of tuples. The first integer in the tuple should be numbers from 0 to 5 (included). The second integer of the tuple should be the square of the first integer.

ANS: new_list = [(index, index**2) for index in range(6)]

Use comprehension to compile a new list, where the elements are odd numbers from 1 to 10 (included)

ANS: new_list = [i for i in range(1,11,2)]

x contains a string, which is a sentence. Print a new string where all the white space between words are removed.

ANS:

x = 'This is a sentence.'

new_list = []

for word in x.split():

new_list.append(word)

new_string = "".join(new_list)

print(new_string)

x contains a string of alphabetic characters. Print a new string where the odd element of x is in upperletter case, while the even elements are in lowerletter case

ANS:

x = 'aNotVeryLongString'

x_list = list(x)

new_list = []

for index, element in enumerate(x):

if index % 2 != 0:

new_list.append(element.upper())

else:

new_list.append(element.lower())

new_string = "".join(new_list)

print(new_string)

x contains a string, which a sentence. Print a list where the odd elements are indices of words and the even elements are words from the string.

x = 'This is a sentence.'

new_list = []

for index, word in enumerate(x.split()):

new_list.append(index)

new_list.append(word)

Keyword	Description	Code example
False, True	Data values from the data type Boolean	False == (1 > 2), True == (2 > 1)
and, or, not	Logical operators: (x or y) → both x and y must be True (x and y) → either x or y must be True (not x) → x must be false	x, y = True, False (x or y) == True # True (x and y) == False # True (not y) == True # True
break	Ends loop prematurely	while(True): break # no infinite loop print("hello world")
continue	Finishes current loop iteration	while(True): continue print("43") # dead code
class	Defines a new class → a real-world concept (object oriented programming)	class Beer: def _init_(self): self.content = 1.0 def drink(self): self.content = 0.0
def	Defines a new function or class method. For latter, first parameter ("self") points to the class object. When calling class method, first parameter is implicit.	becks = Beer() # constructor - create class becks.drink() # beer empty: b.content == 0
if, elif, else	Conditional program execution: program starts with "if" branch, tries the "elif" branches, and finishes with "else" branch (until one branch evaluates to True).	x = int(input("your value: ")) if x > 3: print("Big") elif x == 3: print("Medium") else: print("Small")
for, while	# For loop declaration for i in [0,1,2]: print(i)	# While loop - same semantics j = 0 while j < 3: print(j) j = j + 1
in	Checks whether element is in sequence	42 in [2, 39, 42] # True
is	Checks whether both elements point to the same object	y = x = 3 x is y # True [3] is [3] # False
None	Empty value constant	def f(): x = 2 f() is None # True
lambda	Function with no name (anonymous function)	(lambda x: x + 3)(3) # returns 6
return	Terminates execution of the function and passes the flow of execution to the caller. An optional value after the return keyword specifies the function result.	def incrementor(x): return x + 1 incrementor(4) # returns 5

Example	Result
list(map(lambda x: x[0], ['red', 'green', 'blue']))	['r', 'g', 'b']
list(map(lambda x, y: str(x) + ' ' + y + 's', [0, 2, 2], ['apple', 'orange', 'banana']))	['0 apples', '2 oranges', '2 bananas']
'marries'.join(list('Alice', 'Bob'))	'Alice marries Bob'
list(filter(lambda x: True if x>17 else False, [1, 15, 17, 18]))	[18]
print("\n\t 42\t\t.strip()")	42
sorted([8, 3, 2, 42, 5])	[2, 3, 5, 8, 42]
sorted([8, 3, 2, 42, 5], key=lambda x: 0 if x==42 else x)	[42, 2, 3, 5, 8]
help(str.upper())	'... to uppercase.'
list(zip(['Alice', 'Anna'], ['Bob', 'Anna', 'Jon']))	[('Alice', 'Bob'), ('Anna', 'Jon')]
list(zip*(['Alice', 'Bob'], ('Anna', 'Jon')))	[('Alice', 'Anna'), ('Bob', 'Jon')]
list(enumerate(['Alice', 'Bob', 'Jon']))	[(0, 'Alice'), (1, 'Bob'), (2, 'Jon')]
T	python -m http.server
R	<P>
I	Want to share files between PC and phone? Run this command in PC's shell: <P> is any port number 0-65535. Types < IP address of PC> <P> in the phone's browser. You can now browse the files in the PC directory.
C	Read comic
K	import antigravity
S	Open the comic series xkcd in your web browser
	'...Beautiful is better than ugly. Explicit is ...'
	a, b = 'Jane', 'Alice'
	a, b = b, a
	def f(x, y, z): return x + y * z
	f(*[1, 3, 4])
	f(**{'z': 4, 'x': 1, 'y': 3})
	a, *b = [1, 2, 3, 4, 5]
	a = 1 b = [2, 3, 4, 5]
	x = {'Alice': 18}
	y = {'Bob': 27, 'Ann': 22}
	z = {**x, **y}

Write a function that take an unlimited number of arguments. The function should return a list of numbers that are divisible by 2

```
def main(*args):
    return [i for i in args if type(i) == int and i % 2 == 0]
```

x contains a list of integers. Use comprehension to compile a new dictionary, where the keys are elements of x that are divisible by 2, and the values are the squares of those elements

x = [1, 2, 3, 4]

new_list = {num: num ** 2 for num in x if num % 2 == 0}